# BSDA2001P: Introduction to DL and GenAI Project

Submitted by

Name: Sharmila M

Roll number: 24ds2000048

IITM Online BS Degree Program,

Indian Institute of Technology, Madras, Chennai Tamil Nadu, India, 600036

# Contents


**1. Executive Summary** ..... 2

**2. Introduction** ..... 2

**2.1 Problem Statement** ..... 2

**2.2 Project Objectives** ..... 2

**2.3 Report Structure** ..... 2

**3. Dataset & Preprocessing** ..... 3

**3.1 Dataset Description** ..... 3

**3.2 Exploratory Data Analysis (EDA)** ..... 3

**3.3 Data Preprocessing** ..... 4

**4. Tokenization Strategy** ..... 4

**4.1 Primary Tokenizer Selection** ..... 4

**4.2 Justification for Tokenization Choice** ..... 4

**4.3 Implementation Details** ..... 4

**4.4 Experimental Comparison with Alternative Tokenization** ..... 4

**5. Modeling & Experimentation** ..... 5

**5.1 BiLSTM Baseline (Custom Deep Learning Model)** ..... 5

**5.2 BERT-Based Model (Fine-Tuned Transformer 1)** ..... 5

**5.3 RoBERTa-Based Model (Fine-Tuned Transformer 2)** ..... 6

**6. Performance & Comparative Analysis** ..... 7

**6.1 Evaluation Metrics** ..... 7

**6.2 Training Dynamics and Model Behaviour** ..... 7

**7. Conclusion & Future Work** ..... 8

**7.1 Key Learnings** ..... 9

**7.2 Challenges Faced** ..... 9

**7.3 Future Work** ..... 9

**8. References** ..... 9

# 1. Executive Summary

This project tackles a multi-label emotion classification task from a Kaggle competition, where each short text can express one or more of five emotions: ***anger, fear, joy, sadness, and surprise.*** The goal is to build models that predict the correct set of emotions for each input text and achieve a strong macro-F1 score on the leaderboard.

Three deep learning approaches were implemented and compared: a BiLSTM baseline trained from scratch, and two transformer-based models (BERT base and RoBERTa base) fine-tuned for this task. After data exploration, careful preprocessing, and model tuning, the best model (RoBERTa) achieved high macro-F1 on the validation set and competitive performance on Kaggle. The experiments highlight the benefits of pretrained transformers over recurrent models.

# 2. Introduction

## 2.1 Problem Statement

The competition provides a dataset of short, user-generated texts, each annotated with zero or more emotion labels from the set {anger, fear, joy, sadness, surprise}. The task is framed as multi-label text classification: for each text, the model must output a binary vector indicating which emotions are present. The official evaluation metric is macro-averaged F1 score, which averages the F1 scores of the five labels and therefore emphasizes balanced performance across both frequent and rare emotions.

## 2.2 Project Objectives

The primary goal of this project is to

- Explore different neural architectures, ranging from simple feed-forward networks to sequence-based and transformer-based models.
- Compare the effectiveness of custom-built models versus pretrained language models.
- Gain practical experience with modern NLP frameworks such as PyTorch, Hugging Face Transformers, and experiment tracking tools.
- Deploy the best-performing model as an interactive application to demonstrate real-world usability.

## 2.3 Report Structure

The report first describes the dataset, exploratory analysis, and preprocessing steps. It then explains the tokenization strategies and details the architecture and training procedures for each model. Next, it presents the performance comparison and discusses key findings. Finally, it concludes with lessons learned and suggestions for future work.

# 3. Dataset & Preprocessing

## 3.1 Dataset Description

The training set contains ***6827*** labeled texts, and the test set contains ***1707*** unlabeled texts. Each row consists of an id, the text, five binary columns for the emotions, and a derived list of active emotions. The label distribution is imbalanced: fear is the most frequent emotion, followed by sadness, surprise, joy, and anger, which is the least represented. Many samples contain multiple emotions simultaneously, and a non-trivial number have no active emotion label at all, which adds difficulty to the task.

## 3.2 Exploratory Data Analysis (EDA)

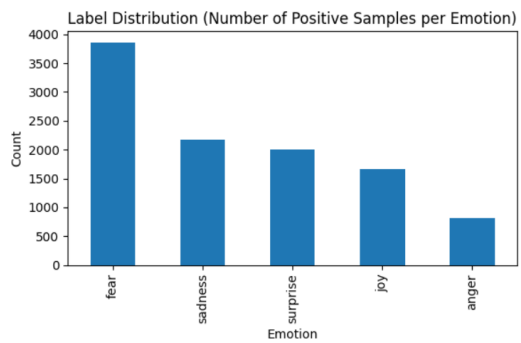


***Figure 1:*** *Distribution of emotion labels*

The training data is clearly imbalanced: fear is most common with **3,860** samples, followed by sadness **(2,171),** surprise **(1,999),** joy **(1,660),** and anger, which is least frequent with **808** samples(Refer Figure 1)

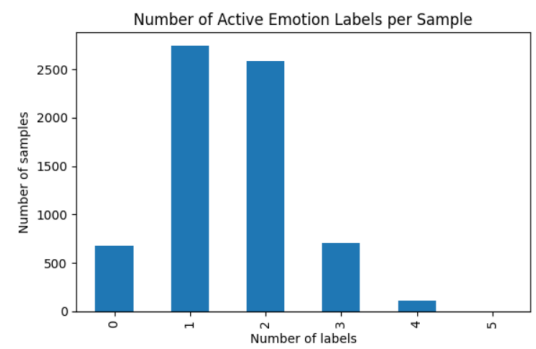


***Figure 2:*** *Number of active emotion labels per sample*

The training set contains **2,743** samples with one active emotion, **2,587** with two, **706** with three, a small number with four or five, and **676** with no active emotion labels, which increases task complexity. (Refer Figure 2)

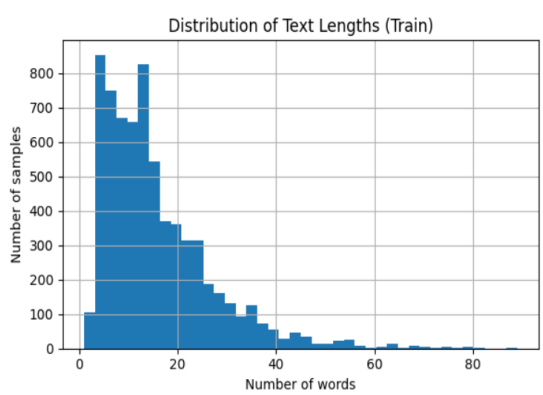


***Figure 3:*** *Distribution of text lengths (word count) in the training dataset*

The textual inputs in the dataset are relatively short but variable in length. An analysis of word counts per sample shows that training texts have an average length of approximately **15.7 words**, with a median length of **13 words**. The shortest texts contain a single word, while the longest training sample contains **89 words**. (Refer Figure 3)

These findings guided downstream modeling decisions, including the selection of a **maximum sequence length of 80 tokens** for transformer-based models, which ensures that most texts are fully captured without excessive truncation.

### 3.3 Data Preprocessing

For the BiLSTM baseline, the text is lowercased and tokenized using simple ***whitespace*** splitting. A vocabulary is built from the training texts with special tokens <PAD> and <UNK>. Each text is converted to a sequence of word indices and padded or truncated to a fixed length of 50 tokens.

For the transformer models, the raw text is passed directly to the pretrained tokenizers which uses subword tokenization and automatically handle lowercasing, special tokens, and mapping to input IDs and attention masks. The maximum sequence length is set to 80 tokens, which covers almost all examples without excessive padding.

## 4. Tokenization Strategy

### 4.1 Primary Tokenizer Selection

The primary tokenization strategy for the best-performing models is subword tokenization from the Hugging Face implementations of BERT and RoBERTa. BERT uses ***WordPiece*** tokenization with a 30k-word vocabulary learned on large English corpora, while RoBERTa uses ***byte-pair encoding (BPE)*** with its own vocabulary. Both tokenizers split rare or unseen words into smaller subword units, allowing the models to handle misspellings, slang, and morphological variants without an exploding vocabulary.

### 4.2 Justification for Tokenization Choice

Subword tokenization is well-suited to this task because the dataset contains informal, user-generated text with many rare words and creative spellings. Representing such words as combinations of frequent subwords leads to better coverage than a pure word-level vocabulary and leverages the rich contextual embeddings learned during pretraining. Using the official tokenizers also ensures that the inputs are compatible with the corresponding pretrained checkpoints.

### 4.3 Implementation Details

Tokenization is implemented using the Hugging Face ***__call__*** interface with max_length=80, padding="max_length", and truncation=True. Texts are batched in PyTorch DataLoaders to exploit parallelism on the GPU. Attention masks are used so that the transformers ignore padded positions. For the BiLSTM model, a custom word-level tokenizer and index mapping are implemented, followed by PyTorch's ***nn.Embedding*** for learned word vectors.

### 4.4 Experimental Comparison with Alternative Tokenization

An initial baseline used only the BiLSTM's simple whitespace tokenization and fixed vocabulary. Subsequent experiments with BERT and RoBERTa showed substantial

performance gains, validating the choice of pretrained tokenizers and highlighting the importance of rich subword representations for emotion classification.

# 5. Modeling & Experimentation

## 5.1 BiLSTM Baseline (Custom Deep Learning Model)

### *5.1.1 Architecture Overview*

The first model is a custom bidirectional LSTM (BiLSTM) network implemented from scratch in PyTorch. The architecture(***Figure 4***) directly follows the training code:

- An nn.Embedding layer maps word indices from the custom vocabulary to 128-dimensional embeddings.
- A single nn.LSTM layer with hidden size 64 and bidirectional=True processes the sequence and produces forward and backward hidden states.
- The final hidden state from the forward direction and the final hidden state from the backward direction are concatenated into a 128-dimensional sentence representation.
- A fully connected layer projects this representation to five outputs, one per emotion, followed by a sigmoid activation to obtain independent label probabilities.

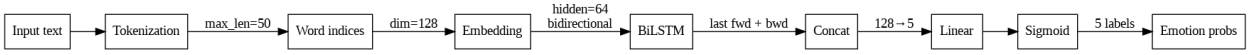


***Figure 4****: BiLSTM Architecture*

### *5.1.2 Training Strategy*

The model is trained with:

- **Loss**: BCELoss over the five sigmoid outputs.
- **Optimizer**: Adam with learning rate 1e-3.
- **Batch size**: 64; maximum sequence length: 50 tokens.
- Early stopping on validation macro-F1 with patience 3, using a 10% validation split and threshold 0.5 to binarize probabilities.

This BiLSTM serves as a strong from-scratch baseline to compare against pretrained transformers.

## 5.2 BERT-Based Model (Fine-Tuned Transformer 1)

### *5.2.1 Architecture Overview*

The second model is built on top of bert-base-uncased(***Figure 5***) from Hugging Face. The code uses:

- BertTokenizer to convert raw text into WordPiece token IDs and attention masks with max_length=80.
- BertModel as an encoder, producing contextual token embeddings and a pooled output.
- A dropout layer with rate 0.3 on the pooled output.
- A linear classification head mapping from the 768-dimensional pooled vector to five logits (one per emotion).
- A sigmoid applied at inference time to convert logits to probabilities.

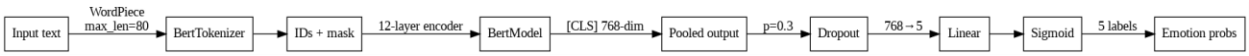


***Figure 5**: Bert Architecture*

### *5.2.2 Training Strategy*

All BERT parameters are fine-tuned jointly with the classification head.

- **Loss**: BCEWithLogitsLoss (logits + sigmoid inside the loss).
- **Optimizer**: AdamW with learning rate 2e-5.
- **Batch size**: 16, maximum of 25 epochs with early stopping (patience 3) based on validation macro-F1.
- **Validation split**: 10% of the training data; threshold 0.5 for converting probabilities to binary labels.

Weights & Biases is used to log train_loss, val_f1_macro, and val_accuracy each epoch. After training, the best checkpoint is saved and uploaded to KaggleHub, then reloaded in a separate inference script to generate test predictions and the final Kaggle submission file.

## 5.3 RoBERTa-Based Model (Fine-Tuned Transformer 2)

### *5.3.1 Architecture Overview*

The third model uses roberta-base(***Figure 6***) as the encoder, with a structure parallel to the BERT model:

- RobertaTokenizer converts text into BPE subword IDs and attention masks, again with max_length=80.
- RobertaModel encodes the sequence and returns a pooled representation.
- A dropout layer (0.3) is applied to the pooled output.
- A linear classifier projects the 768-dimensional pooled vector to five logits.
- A sigmoid at inference time yields multi-label emotion probabilities.

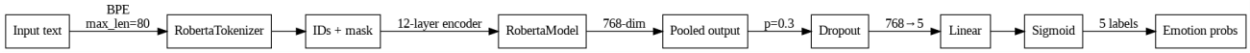


***Figure 6**: Roberta Architecture*

### *5.3.2 Training Strategy*

The training loop mirrors the BERT setup:

- **Loss**: BCEWithLogitsLoss.
- **Optimizer**: AdamW with learning rate 2e-5.
- **Batch size**: 16, maximum of 5 epochs with early stopping (patience 3) on validation macro-F1.
- 10% of the training data is held out for validation; threshold 0.5 is used for binarization.
- Metrics and loss are logged to Weights & Biases each epoch.

## 6. Performance & Comparative Analysis

### 6.1 Evaluation Metrics

The primary metric is ***macro-averaged F1 score***, matching the Kaggle competition's scoring function. This metric separately computes F1 for each emotion and averages them, ensuring that performance on minority labels like anger is reflected fairly. Accuracy is also reported as a secondary metric but is less informative due to label imbalance and the multi-label setup.

### 6.2 Training Dynamics and Model Behaviour

Figure 7 shows the training loss, validation macro-F1, and validation accuracy for all three models (BiLSTM baseline, BERT, and RoBERTa) logged in wandb. The BiLSTM's loss decreases steadily but plateaus at a lower macro-F1, while both transformers quickly reach much higher macro-F1 values. The transformer loss curves show faster convergence and lower final loss, indicating better fit to the data.

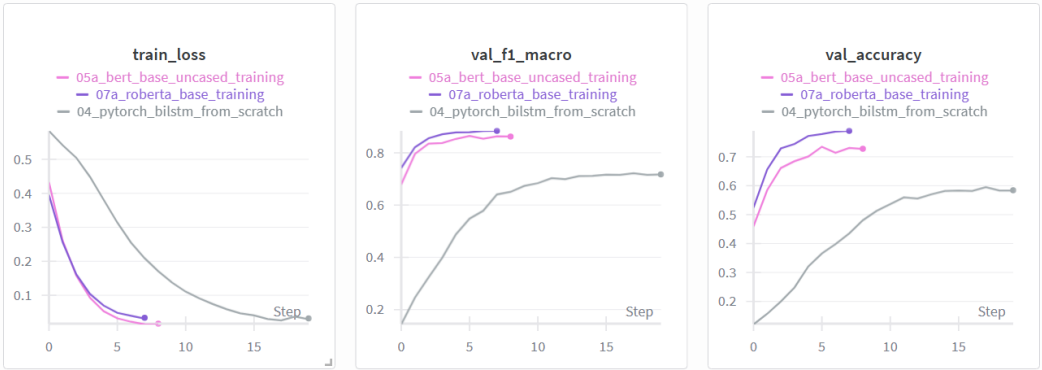


***Figure 7**: wandb logs of 3 selected models*

In the train_loss plot, the BiLSTM (grey) starts with the highest loss and decreases slowly over many steps, while BERT (magenta) and RoBERTa (purple) converge much faster to significantly lower loss values. This indicates that the pretrained transformers fit the data more effectively and require fewer epochs to learn useful representations. The BiLSTM, initialized from scratch, needs more updates and still cannot match the final training loss of the transformers.

The val_f1_macro curves show an even clearer gap in generalization performance. Both BERT and RoBERTa quickly rise to macro-F1 values above 0.8, whereas the BiLSTM climbs gradually and plateaus around the mid-0.7 range. Among the transformers, RoBERTa reaches the highest macro-F1 at the end of training, with BERT just slightly below it, confirming that pretrained contextual embeddings are crucial for capturing multiple overlapping emotions in short texts.

A similar pattern appears in the val_accuracy plot: transformer accuracies rapidly exceed 0.7, while the BiLSTM lags behind around 0.6 despite longer training. ***Table 1*** summarizes the performance of the three selected models in Kaggle.

| Model | Validation Macro F1 | Validation Accuracy | Kaggle Macro F1 |
|---|---|---|---|
| **BiLSTM (from scratch)** | ~0.72 | ~0.58 | 0.714 |
| **BERT (bert-base-uncased)** | ~0.85 | ~0.72 | 0.854 |
| **RoBERTa base** | ~0.88 | ~0.78 | 0.877 |

**Table 1:** Performance Comparison of Selected Models in kaggle

Taken together, these logs show that (i) moving from a from-scratch recurrent model to pretrained transformers yields a large improvement in both learning speed and final performance, and (ii) on this specific dataset and hyperparameter setting, RoBERTa marginally outperforms BERT, although their curves are very close, so RoBERTa transformer is a strong choice compared to the BiLSTM baseline.

## 7. Conclusion & Future Work

The fine-tuned ***RoBERTa base*** model significantly outperformed other approaches, achieving a ***Kaggle Macro F1-score of 0.877,*** and showed strong generalization indicated by close alignment between validation and leaderboard performance. This progression highlights the effectiveness of pretrained transformer models for emotion classification tasks, particularly in scenarios where emotions are subtle and overlapping.

### 7.1 Key Learnings

The project provided several important insights:

- Pretrained language models dramatically reduce training time while improving performance.
- Macro F1-score is a more reliable evaluation metric than accuracy for multi-label, imbalanced datasets.
- Experiment tracking tools such as Weights & Biases (wandb) are essential for systematic model comparison and reproducibility.
- Custom models built from scratch, such as BiLSTM networks, remain valuable for understanding core deep learning concepts despite being outperformed by transformers.

### 7.2 Challenges Faced

Key challenges encountered during the project include:

- Handling class imbalance across emotion labels.
- Selecting appropriate thresholds for multi-label classification.
- Managing computational constraints during transformer fine-tuning.
- Ensuring reproducibility across experiments and leaderboard submissions.

These challenges were addressed through careful metric selection, early stopping, experiment tracking, and controlled training procedures.

### 7.3 Future Work

Several extensions could further improve performance and robustness:

- Exploring larger or more advanced transformer architectures such as DeBERTa or RoBERTa-large.
- Applying advanced data augmentation techniques or ensemble strategies.
- Investigating threshold tuning per emotion class rather than a global threshold.
- Deploying the model as a scalable API with monitoring for real-world usage.

## 8. References


1. Devlin, J., Chang, M.-W., Lee, K., & Toutanova, K. (2019).
   *BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding.*
   Proceedings of NAACL-HLT. https://arxiv.org/abs/1810.04805
2. Hochreiter, S., & Schmidhuber, J. (1997).
   *Long Short-Term Memory.* Neural Computation, 9(8), 1735–1780.
3. Project Repo - GitHub Link
4. Hugging Face deployed app link - Hugging Face App link